ПРОТОКОЛ ОБМЕНА: БОРТОВОЙ ПК ↔ КОНТРОЛЛЕРЫ КОЛЁС

Единственный протокол робота — **текстовый, построчный, по USB-CDC/UART**. Никаких бинарных заголовков, кадров фиксированной длины и CAN/RS-485 в проекте нет: каждая плата Arduino Mega 2560 / Mega Mini подключена к бортовому компьютеру **своим USB-кабелем** и обменивается строками, заканчивающимися переводом строки.

Источник истины — скетч firmware/motor_wheel/motor_wheel.ino (текущая версия **fw25**). Документ описывает то, что реально делает прошивка; описание команд монитора и калибровок — в firmware/README.md.

Параметр	Значение
Физика	USB (CDC) или UART, **115200 8N1**
Кодировка	ASCII, кириллица только в человекочитаемых строках # … (UTF-8)
Разделитель команд	n (монитор порта: «NL» или «NL & CR»)
Одна плата = одно колесо	`MODULE_ID` = FL / FR / RL / RR
Роль в телеметрии	поле `mod=` (`@TLM`) и 2-е поле `@F` — цифровой код 1…4
Контроль целостности	**CRC16/Modbus** (полином `0xA001`, init `0xFFFF`), hex-суффикс `crc=` / 4 hex-цифры
Сторож привода	нет команды тяги **650 мс** → выход 0 и тормоз
Короткая телеметрия	`@F` каждые ~30 мс (не чаще 20 мс)
Полная телеметрия	`@TLM` каждые 200 мс
Ответ на запись нуля	`@Z` (только после `zq`/`zc`)

---

1. Направление ПК → контроллер: строки команд

Команда — обычная текстовая строка, например `w FL 12.5`. Сервер всегда шлёт **адресную** форму: имя колеса присутствует, а сама плата получает уже короткую строку без имени (имя удаляется сервером до отправки). В мониторе порта видны обе формы: прошивка принимает своё имя и **игнорирует чужое**.

1.1. Руль

Команда	Назначение
`w [XX] <угол>`	абсолютный угол колеса, °; нужен заданный ноль; предел **±100°**
`w [XX] zero`	ручной ноль в текущем положении
`w [XX] calib`	калибровка по оптодатчику D18
`w [XX] z` / `w [XX] k`	короткие формы `zero` / `calib`
`k` / `calib`	калибровка **всех** колёс (рассылается на все порты)
`z` / `zero`	ручной ноль без имени
`h`	в ноль
`d <угол> [f/b]`	относительный поворот, цель в пределах ±125°
`c <угол>` / `c c`	качание / выключить качание
`g`	продолжить поворот, прерванный `s`
`s`	СТОП всего: руль, калибровка, качание, тяга

1.2. Тяга

Команда	Назначение
`v [XX] <0…150> [f/b]`	прямое значение `Servo.write`, без пересчёта; `f` по умолчанию
`v [XX] 0`	останов и тормоз
`t [XX] <−100…+100>`	прежний процентный режим (1 % ≈ 1,8 значения шкалы)
`b`	тормоз всех BLDC
`b <id>`	тормоз с идентификатором для подтверждения (`brake_id`)
`n <id>`	**только в fw25:** не отпускает тормоз, а вызывает тормоз

1.3. Датчик положения и служебные

Команда	Назначение
`q`	калибровка потенциометра A0: 0°, −60°, +60° (включает доводку)
`pm 1` / `pm 0`	режим «с потенциометром» / «без потенциометра» (сбрасывает `q`)
`ph 1` / `ph 0`	активное удержание угла вкл/выкл (fw24+)
`j <id> <seq> <тяга> <угол>`	служебное руление на ходу (fw20+)
`s q`	тот же STOP, но без обычной заметки о паузе (fw21+)
`zq <id>`	проверка сохранённой поправки полного нуля (fw23+)
`zc <id>`	запись поправки полного нуля в EEPROM (fw23+)
`zr CONFIRM`	сброс неисправной сохранённой поправки
`p`	состояние колеса (человекочитаемый отчёт)
`?` / `help`	справка

1.4. Что проверяет плата до выполнения

Прошивка отклоняет команду **молча в телеметрии, но с текстом в USB**. Правила:

- `w` требует заданного нуля (`cal=1`), иначе `w: ноль не задан — сначала k …`;
- цель `w` вне ±100° отклоняется;
- поворот запрещён, пока включена тяга (`руль заблокирован: включена тяга`);
- смена режима во время движения/калибровки требует `s`;
- `v` требует свежих команд и работает без нуля; `t` требует ноль руля;
- любая команда тяги старше 650 мс → выход 0 и тормоз;
- `ph 1` требует `pm 1`, ноль и выполненную `q`.

---

2. Направление контроллер → ПК

Три типа строк. Первый символ определяет тип: `#` — человеку, `@` — машине.

2.1. Сообщения для человека: `# XX: текст`

```
# FL: --- КОЛЕСО FL: руль v25, USB 115200; ноль не задан — k или z ---
# FL: Калибровка ЗАВЕРШЕНА! Флажок: 12.3°, ноль: 0.0°
# FR: Потенциометр: расхождение 31.08° — доводка отключена; проверьте колесо или q
```

Формат: `# <РОЛЬ>: <UTF-8 текст>\r\n`. Сервер складывает эти строки в журнал экрана/пульта. **Разбирать их программно нельзя** — для логики есть @F/@TLM.

2.2. Короткая телеметрия: @F

Кадр формируется обычно каждые **30 мс**; изменение состояния (тормоз, тяга, руление, свежие фронты SC) запрашивает кадр раньше, но не чаще одного раза в **20 мс**. Длина строки ≤ 160 байт.

```
@F <fw> <role> <deg_c> <tgt_c> <status> <drive> <dmax> <sv> <rpm_x10> <hz> <ppr> <count> <recent> <brake_id> <neutral_id> <neutral_lock> <live_id> <live_seq> <ms> <crc>
```

№	Поле	Тип	Значение
1	fw	u8	версия протокола прошивки, сейчас 25
2	role	u8	1=FL, 2=FR, 3=RL, 4=RR, 0=роль в скетче не задана
3	deg_c	i32	фактический угол, **сотые доли градуса** (1250 = 12.50°)
4	tgt_c	i32	целевой угол, сотые доли градуса
5	status	u16	биты состояния, см. §3
6	drive	i16	заданная тяга, −100…+100 % (шкала Servo)
7	dmax	u8	предел процентной тяги (t), сейчас 100
8	sv	u8	значение, переданное в Servo.write (0…180)
9	rpm_x10	u32	обороты по SC в **десятых** (600 = 60.0 об/мин)
10	hz	u16	принятая частота фронтов SC, Гц
11	ppr	u16	принятых переходов на оборот (TACH_EDGES_PER_REV)
12	count	u32	непрерывный счётчик фронтов SC по модулю 2^{32}
13	recent	u8	1 — фронты были недавно (окно 300…1500 мс)
14	brake_id	u32	подтверждённый id тормоза b N, 0 — нет
15	neutral_id	u32	устаревшее поле нейтрали (fw18…24), в fw25 всегда 0
16	neutral_lock	u8	устаревшая блокировка нейтрали, в fw25 всегда 0
17	live_id	u32	id живой сессии руления на ходу (j), 0 — нет
18	live_seq	u32	последний применённый номер пакета j
19	ms	u32	millis() платы по модулю 2^{32}
20	crc	hex16	CRC16/Modbus предыдущих байт строки

2.3. Полная телеметрия: @TLM

Каждые **200 мс** (совместимость со старыми серверами). Длина ≤ 384 байт. Поля — ключ=значение в фиксированном порядке:

```
@TLM mod=FR deg=12.50 tgt=12.50 moving=0 cal=1 cycle=0 calib=0 opto=1 home=opto
     err=0 drive=0 fw=25 pot=1 pm=1 ph=0 zfault=0 tach=0 dmax=100 sv=0 duty=0
     sc_rpm=0.0 sc_hz=0 sc_ppr=90 sc_count=0 sc_recent=0 brake_id=0 neutral_id=0
     neutral_lock=0 live_id=0 live_seq=0 t=6970 crc=28F7
```

Поле	Значение
mod	роль: FL / FR / RL / RR
deg, tgt	фактический и целевой угол, ° (-135.00…135.00)
moving	1 — руль едет
cal	1 — ноль задан
home	откуда ноль: opto / manual / none
calib	1 — идёт калибровка
opto	1 — датчик D18 сейчас закрыт (колесо над флажком)
cycle	1 — идёт качание c

Поле	Значение
`err`	код ошибки калибровки: 0 — нет; 1…6 см. `firmware/README.md` §5
`drive`	заданная тяга, %
`fw`	версия прошивки/протокола
`pot`	1 — q выполнена, доводка включена
`pm`	режим потенциометра: 1/0, `-1` — прошивка не поддерживает переключение
`ph`	активное удержание угла: 1/0
`zfault`	1 — сохранённая поправка полного нуля повреждена
`tach`	всегда 0 (старый канал одометрии выключен)
`dmax`	предел старой процентной тяги
`sv`	`Servo.write` (0…180)
`duty`	то же в промилле шкалы Servo (1000 = `write(180)`)
`sc_rpm`	обороты по SC, об/мин
`sc_hz`	частота фронтов SC, Гц
`sc_ppr`	переходов на оборот
`sc_count`	счётчик фронтов по модулю 2^{32}
`sc_recent`	1 — фронты были недавно
`brake_id`, `neutral_id`, `neutral_lock`, `live_id`, `live_seq`	подтверждения сессий (см. §2.2)
`t`	`millis()` платы
`crc`	CRC16/Modbus строки до `" crc="`

2.4. Ответ на запись полного нуля: @Z

```
@Z <fw> <role> <request_id> <status> <saved_steps> <proposed_steps> <crc>
```

Поле	Значение
`fw`	версия прошивки (23+)
`role`	1…4
`request_id`	id из `zq <id>` / `zc <id>`
`status`	0 — успех/подтверждение, иначе код ошибки
`saved_steps`	сохранённая поправка в шагах
`proposed_steps`	предложенная поправка в шагах (для `zc`)
`crc`	CRC16/Modbus

Кадр повторяется недолго; после перезагрузки платы id запросов забываются.

3. Биты состояния

Бит	Маска	Смысл
0	0x0001	MOVING — руль едет
1	0x0002	ZERO — ноль задан (cal)
2	0x0004	CALIB — идёт калибровка
3	0x0008	OPTO — датчик D18 закрыт
4	0x0010	CYCLE — качание
5	0x0020	HOME_OPTO — ноль получен по оптодатчику
6	0x0040	V2 — признак кадра этой версии
7	0x0080	POT — потенциометр откалиброван (q), доводка включена
8–11	0x0F00	код ошибки калибровки err (сдвиг 8)
12	0x1000	POT_MODE — режим «с потенциометром»
13	0x2000	POT_MODE_SUPPORTED — прошивка умеет pm
14	0x4000	FULL_ZERO_FAULT — сохранённая поправка не заслуживает доверия
15	0x8000	POT_HOLD — активное удержание угла включено

---

4. CRC16/Modbus

Полином 0xA001, инициализация 0xFFFF, байты — как есть, без инверсии. Считается по **всем байтам строки до суффикса**, то есть без `" crc="` (в @TLM) и без ведущего пробела перед четырьмя hex-цифрами (в @F, @Z).

```
def crc16(data: bytes) -> int:
    crc = 0xFFFF
    for byte in data:
        crc ^= byte
        for _ in range(8):
            crc = (crc >> 1) ^ 0xA001 if crc & 1 else crc >> 1
    return crc
```

Сервер обязан **отбрасывать кадр с неверной CRC** и не обновлять по нему время свежести: иначе битая строка с cal=1 откроет движение. CRC защищает только от случайных ошибок передачи; это **не аутентификация и не E-STOP**.

---

5. Таймауты и частота

Событие	Значение	Реакция
Нет команды тяги	**650 мс**	drive = 0, BRAKE активен, STOP снят
Потеря @F	по настройке экрана (обычно 1 с)	модуль считается офлайн, команды не шлются
@F при изменении состояния	не чаще **20 мс**	срочный кадр
@F в покое	**30 мс**	обычный кадр
@TLM	**200 мс**	совместимость
Таймаут калибровки	120 с	err=5, ноля нет

Скорость линии **115200 бод 8N1** оставлена намеренно: 250000 бод возможны, но требуют согласованной правки обеих сторон и проверки конкретного USB-UART.

---

6. Со стороны ПО (Zapret)

Весь бортовой софт — в соседнем репозитории danilka-revin/Zapret:

- `slam_gui/backend.py` — `MasterSource`: открывает 4 порта, парсит `@F`/`@TLM`, адресует команды по имени колеса, следит за свежестью и подтверждениями;
- `robot_control/rus_core/protocol.py` — сборка/разбор строк протокола и CRC16, общий для сервера и тестов;
- `tests/test_firmware.py` в Zapret читает `FIRMWARE_PROTOCOL` **из этого скетча** и сверяет с сервером; симуляция прошивки (`firmware/firmware_sim` здесь) отдаёт настоящие кадры через PTY.

Порядок изменений: сначала этот документ и скетч, потом парсер и тесты в Zapret.